

OpenKits Studio 项目规划文档

项目背景

目前 OpenKits Skills 已经具备 Agent 能力：

- 工程理解
- 代码生成
- 文件修改
- SDK 调用
- 编译流程控制

但部署门槛较高，需要用户自行安装：

- Node.js
- Git
- OpenKits Agent
- CC Switch
- MSPM0 SDK
- CCS Compiler
- XDS110 驱动
- J-Link 驱动
- AI模型配置

对于开发板用户而言过于复杂。

目标是将整个开发环境封装为一个 Windows 桌面应用，实现：

安装即用（Install & Run）

让用户无需关心环境配置。

产品定位

产品名称：

OpenKits Studio

副标题：

AI Native Embedded Development Studio

定位：

面向嵌入式开发板的 AI IDE。

不是传统 IDE。

而是：

AI Agent + 工程管理 + 编译系统 + 烧录系统

的统一入口。

核心目标

用户安装后：

1. 打开软件
2. 选择开发板
3. 创建工程
4. 与 AI 对话
5. 自动修改代码
6. 一键编译
7. 一键烧录

整个过程无需安装任何额外工具。

第一阶段目标（MVP）

仅支持：

- MSPM0系列开发板

例如：

- 天巧星 MSPM0G3519

支持：

- AI代码修改
- 工程管理
- 编译
- 烧录

暂不追求：

- 多平台
 - 调试器
 - 波形分析
 - RTOS可视化
-

技术架构

整体架构

OpenKits Studio

|— Electron |— React |— Node Runtime |— OpenKits Agent |— Toolchain Manager |— Compiler Manager |— Flash Manager |— AI Provider Manager

前端

技术：

- React
- TypeScript
- Tailwind

界面类似：

VS Code + Cursor

布局：

左侧

- 工程树

中间

- 代码编辑器

右侧

- AI Agent

底部

- 编译日志
 - 烧录日志
-

编辑器

采用：

Monaco Editor

不要自行开发编辑器。

目标：

获得 VS Code 同级体验。

支持：

- 语法高亮
 - 搜索
 - 文件树
 - Diff
-

桌面框架

采用：

Electron

原因：

- 开发快
- Node兼容性好
- 可调用本地程序
- 可读写文件

目标：

Windows优先。

后续：

- macOS
 - Linux
-

Agent系统

核心能力

Agent 可以：

读取工程

修改工程

创建文件

删除文件

执行命令

分析日志

生成代码

Agent权限模型

允许：

ReadFile

WriteFile

CreateFile

DeleteFile

RunCommand

SearchProject

禁止：

访问系统目录

访问用户隐私目录

联网执行未知脚本

Agent workflow

用户：

增加ADC采样功能

↓

Agent

读取工程

↓

分析main.c

↓

修改代码

↓

生成Diff

↓

展示给用户

↓

确认

↓

写入工程

AI模型系统

支持模型

OpenAI

Claude

DeepSeek

Qwen

Gemini

OpenRouter

Provider抽象

```
interface AIProvider {
```

```
  chat()
```

```
  stream()
```

```
  toolCall()
```

}

配置方式

设置页：

API Key

Base URL

Model Name

工程管理系统

创建工程

模板来源：

Examples

SDK Templates

Github Templates

工程结构

Project

|— src |— include |— docs |— build |— .openkits

.openkits

保存：

开发板信息

SDK版本

编译配置

Agent状态

Toolchain Manager

目标：

自动管理工具链。

用户无需安装。

Runtime目录

OpenKitsStudio

|— runtime | |— node | |— git | |— sdk | |— compiler | |— xds110 | |— jlink

启动检查

检查：

Git

Node

Compiler

SDK

JLink

XDS110

缺失：

自动下载。

编译系统

Build Adapter

统一接口：

```
interface Builder {
```

```
    build()
```



```
clean()

rebuild()

}
```

第一阶段

支持：

MSPM0 CCS

编译流程

点击编译

↓

调用 Builder

↓

输出日志

↓

解析错误

↓

交给 Agent

↓

生成修复建议

烧录系统

Flash Adapter

```
interface Flasher {

detect()
```

flash()

verify()

}

第一阶段支持

XDS110

J-Link

自动识别

插入设备

↓

识别调试器

↓

显示：

XDS110

或

J-Link

↓

允许烧录

AI辅助功能

功能1

解释代码

用户：

解释这个工程

Agent:

输出模块分析

功能2

新增功能

用户:

增加ADC采样

Agent:

自动改代码

功能3

修复编译错误

用户:

点击编译

↓

失败

↓

日志发送给Agent

↓

输出修复方案

功能4

生成驱动

用户:

生成I2C OLED驱动

Agent:

自动创建文件

SDK管理

目标：

自动安装SDK。

用户无需手动配置。

支持

MSPM0 SDK

后续：

STM32

ESP32

K230

RK3566

CH32

GD32

开发板生态规划

未来每块开发板拥有：

Board Package

包含：

- PinMap
 - Example
 - SDK配置
 - 烧录配置
 - 文档
-

Board Package结构

board.json

pinmap.json

examples/

docs/

flash.json

MVP开发顺序

Phase 1

Electron框架

Monaco编辑器

文件树

工程打开

Phase 2

OpenKits Agent接入

文件修改

Diff预览

Phase 3

MSPM0 SDK集成

CCS编译器接入

编译日志

Phase 4

XDS110

J-Link

烧录支持

Phase 5

模型配置系统

OpenAI

DeepSeek

Claude

Phase 6

安装包

一键安装

环境检测

自动修复

MVP验收标准

用户下载：

OpenKitsStudioSetup.exe

安装完成

↓

创建 MSPM0 工程

↓

对 AI 说：

增加ADC采样功能

↓

Agent自动修改代码

↓

点击编译

↓

编译成功

↓

点击烧录

↓

下载到开发板

↓

运行成功

达成 MVP。